

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Scenario-based

1. Secure Data Transmission using Block Cipher

A company is transmitting repeated financial data blocks over a network. Initially, they used ECB mode and noticed patterns in ciphertext.

- A. Explain why this issue occurs.
- B. Suggest a more secure block cipher mode and justify your choice based on the scenario.

2. Choosing an Encryption Algorithm for Banking System

A banking application needs high security and moderate performance for encrypting customer transactions. The available options are DES, 3-DES, and

Blowfish.

- A. Identify the most suitable algorithm for this scenario.
- B. Justify your choice based on security strength and efficiency.

3. Key Exchange in an Unsecured Network

Two users want to establish a secure communication channel over a public network using Diffie-Hellman key exchange. However, the network is vulnerable to attacks.

- A. Explain how the key exchange works in this scenario.
- B. Identify a possible security threat and suggest a solution to mitigate it.

4. Public Key Cryptography for Email Security

An organization plans to implement RSA for secure email communication.

- A. Describe how RSA can be used for encryption and decryption in this scenario.
- B. Explain how key distribution and management play a role in maintaining security.

5. Cryptography for IoT Devices

A smart home system uses low-power IoT devices that need secure communication. The designer must choose between RSA and Elliptic Curve Cryptography (ECC).

- A. Analyze the constraints of IoT devices.
- B. Recommend a suitable cryptographic approach and justify your answer.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
----------	---------------	-------------------------	-------------------------	--------------------------	------------------------

Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

		presented			
--	--	-----------	--	--	--

ANSWERS – ASSESSMENT TOOL 2

Scenario-Based – Cryptography and Network Security

Q1. Secure Data Transmission Using Block Cipher

Scenario Analysis

The company is transmitting financial data containing repeated plaintext blocks. When **Electronic Codebook (ECB)** mode is used, patterns become visible in the ciphertext, which can reveal information about the structure of the transmitted data.

A. Why Does This Issue Occur?

In ECB mode, the plaintext is divided into fixed-size blocks, and every block is encrypted independently using the same encryption key.

The operation can be represented as:

$$C_i = EK(P_i) \quad C_j = E_K(P_j) \quad C_i = EK(P_i)$$

where:

- P_i = plaintext block
- C_i = ciphertext block
- K = encryption key

The major problem is:

$P_1 = P_2 \Rightarrow C_1 = C_2$

For example, assume repeated financial data contains:

$P_1 = 1010$ $P_2 = 1010$

With the same key, ECB produces:

$C_1 = X$ $C_2 = X$

Therefore, an attacker may identify repeated ciphertext patterns even without knowing the plaintext.

This is particularly dangerous for financial information containing repeated transaction structures, headers, fields, or values.

B. More Secure Block Cipher Mode

A better choice than ECB for the given scenario is **CBC (Cipher Block Chaining)**.

In CBC:

$C_i = E_K(P_i \oplus C_{i-1})$

For the first block:

$C_1 = E_K(P_1 \oplus IV)$

where IV is the Initialization Vector.

Each plaintext block is XORed with the previous ciphertext block before encryption.

Therefore, identical plaintext blocks do not generally produce identical ciphertext blocks.

Comparison

Feature	ECB	CBC
Blocks encrypted independently	Yes	No
Uses IV	No	Yes
Repeated patterns visible	Yes	Generally hidden

Chaining between blocks	No	Yes
Suitable for repeated financial data	Poor	Better

Recommendation

CBC is more secure than ECB for hiding repeated patterns\boxed{\text{CBC is more secure than ECB for hiding repeated patterns}}CBC is more secure than ECB for hiding repeated patterns

For a newly designed real-world financial system, an **authenticated-encryption mode such as AES-GCM** would normally be preferred because it provides both confidentiality and integrity/authentication.

Result

ECB reveals patterns because identical plaintext blocks generate identical ciphertext blocks. CBC improves confidentiality by chaining blocks together, while modern authenticated encryption provides even stronger protection.

Q2. Choosing an Encryption Algorithm for Banking System

Scenario Analysis

The banking application requires:

- High security
- Moderate performance
- Protection of customer transactions

Available algorithms:

1. DES
 2. 3-DES
 3. Blowfish
-

A. Most Suitable Algorithm

Among the three given choices:

Blowfish\boxed{\text{Blowfish}}Blowfish

is the most reasonable selection for this classroom comparison.

B. Justification

DES

DES uses an effective key size of:

56 bits\boxed{56\ bits}56 bits

Its small key space makes it vulnerable to brute-force attacks.

Therefore:

DES is not suitable for modern banking security\boxed{\text{DES is not suitable for modern banking security}}DES is not suitable for modern banking security

3-DES

3-DES improves on DES by applying DES operations multiple times.

It provides stronger security than DES, but its major disadvantage is poor performance compared with newer designs.

It is also a legacy algorithm and should not be selected for a new banking application.

Blowfish

Blowfish supports variable key sizes up to 448 bits and was designed as a faster alternative to older encryption systems.

Compared with the other two options in this question, it offers a better balance between security strength and computational efficiency.

Comparison

Parameter	DES	3-DES	Blowfish
Key size	56-bit	Multiple DES keys	32–448 bits
Security today	Weak	Legacy	Stronger than DES in this comparison
Performance	Moderate	Slow	Faster
Suitable among given options	No	Limited	Best choice

Decision

Among the options provided:

Blowfish is preferred\boxed{\text{Blowfish is preferred}}Blowfish is preferred

because it avoids DES's very small key and the significant performance overhead associated with 3-DES.

However, a modern banking system should generally use an established modern standard such as **AES-256**, preferably in an authenticated-encryption mode such as AES-GCM, rather than deploying DES, 3-DES, or Blowfish in a new design.

Result

For the assessment scenario:

Recommended = Blowfish\boxed{\text{Recommended = Blowfish}}Recommended = Blowfish

For a modern real-world banking system:

Recommended = AES\boxed{\text{Recommended = AES}}Recommended = AES

Q3. Key Exchange in an Unsecured Network

Scenario Analysis

Two users, **User A** and **User B**, need to establish a common secret key over a public network.

Diffie-Hellman allows them to establish a shared secret without directly transmitting that secret itself.

A. How Diffie-Hellman Works

First, both users agree publicly on:

$p = \text{large prime number}$

and

$g = \text{generator}$

These values do not need to remain secret.

Step 1 – Private Keys

User A selects private key:

aaa

User B selects private key:

bbb

These values remain secret.

Step 2 – Public Keys

User A calculates:

$A = g^a \bmod p$

User B calculates:

$B = g^b \bmod p$

They exchange AAA and BBB through the public network.

Step 3 – Shared Secret

User A calculates:

$$K = B \bmod p \quad K = B^a \bmod p \quad K = B \bmod p$$

User B calculates:

$$K = A \bmod p \quad K = A^b \bmod p \quad K = A \bmod p$$

Both obtain:

$$K = g^{ab} \bmod p \quad \boxed{K = g^{ab} \bmod p} \quad K = g^{ab} \bmod p$$

Thus:

$$K_A = K_B \quad \boxed{K_A = K_B} \quad K_A = K_B$$

This common value can then be used, typically through a key-derivation function, to derive symmetric encryption keys.

Communication Flow

User A $\rightarrow$ A User B $\leftarrow$ B
User A $\rightarrow$ A User B $\leftarrow$ B
User A $\rightarrow$ A User B $\leftarrow$ B
User A $\rightarrow$ A User B $\leftarrow$ B

Both sides independently derive the same secret.

B. Possible Threat

A major security threat is a:

Man-in-the-Middle (MITM) Attack $\boxed{\text{Man-in-the-Middle (MITM) Attack}}$ Man-in-the-Middle (MITM) Attack

Basic Diffie-Hellman does not authenticate the parties.

An attacker, Eve, could position herself between the two users:

$$A \leftrightarrow \text{Eve} \leftrightarrow B \quad \longleftarrow \text{Eve} \longrightarrow B \quad A \leftrightarrow \text{Eve} \leftrightarrow B$$

Eve intercepts User A's public value and replaces it with her own.

She can also intercept User B's public value and replace it.

Consequently:

$A \leftrightarrow Eve \leftarrow A \leftrightarrow Eve$

may establish one secret, while:

$Eve \leftrightarrow B \leftarrow Eve \leftrightarrow B$

establishes another.

The users may incorrectly believe they are communicating directly.

Solution

The exchange must be authenticated using mechanisms such as:

- Digital signatures
- Digital certificates
- Public Key Infrastructure (PKI)
- Authenticated Diffie-Hellman
- TLS with proper certificate validation

A digital signature can prove that the exchanged public parameters actually belong to the expected sender.

Result

Diffie-Hellman provides an effective method for establishing a shared secret over an insecure network, but **authentication is essential** for preventing Man-in-the-Middle attacks.

Q4. Public Key Cryptography for Email Security

Scenario Analysis

The organization wants to use **RSA** to protect email communication.

RSA uses two mathematically related keys:

Public Key\boxed{\text{Public Key}}Public Key

and

Private Key\boxed{\text{Private Key}}Private Key

The public key can be distributed, whereas the private key must remain confidential.

A. RSA Encryption and Decryption

Assume **User A** wants to send confidential information to **User B**.

Step 1 – Key Generation

User B generates:

Public Key Public Key_B Public Key B

and

Private Key Private Key_B Private Key B

User B distributes the public key but protects the private key.

Step 2 – Encryption

User A obtains User B's authentic public key.

Conceptually:

$C = E_{\text{Public}_B}(M)$ $C = E_{\text{Public}_B}(M)$ $C = E_{\text{Public}_B}(M)$

where:

- M = message
- C = ciphertext

Step 3 – Transmission

User A sends the ciphertext:

$A \rightarrow C \xrightarrow{B} A \leftarrow C$

An interceptor should not be able to recover the message without the appropriate private key.

Step 4 – Decryption

User B uses the private key:

$$M = D_{\text{Private}_B}(C) \quad M = D_{\{\text{Private}_B\}}(C) \quad M = D_{\text{Private}_B}(C)$$

Thus the original information is recovered.

Practical Email Implementation

RSA is inefficient for encrypting large email messages directly.

Therefore, practical systems use **hybrid encryption**.

First, a random symmetric session key KKK is generated.

The email is encrypted using the symmetric key:

$$C_{\text{email}} = E_K(M) \quad C_{\{\text{email}\}} = E_K(M) \quad C_{\text{email}} = E_K(M)$$

Then the session key is protected using the recipient's RSA public key:

$$CK = \text{RSAPublic}_B(K) \quad C_K = \text{RSA}_{\{\text{Public}_B\}}(K) \quad CK = \text{RSAPublic}_B(K)$$

The recipient uses the RSA private key to recover KKK, then uses KKK to decrypt the email.

This combines:

RSA security for key transport + symmetric encryption efficiency
 $\boxed{\text{RSA security for key transport + symmetric encryption efficiency}}$
RSA security for key transport + symmetric encryption efficiency

RSA signatures may also be used to authenticate the sender and protect message integrity.

B. Key Distribution and Management

Key management is extremely important because RSA security depends on the authenticity and protection of cryptographic keys.

Public Key Distribution

Users need a trustworthy way to determine that a public key actually belongs to the intended

recipient.

Digital certificates and a **Public Key Infrastructure (PKI)** can associate identities with public keys.

A Certificate Authority can digitally sign certificates containing public-key information.

Private Key Protection

Private keys must:

- Never be publicly shared
- Be stored securely
- Be protected against unauthorized access
- Use strong authentication
- Be replaced/revoked if compromised

Key Lifecycle

Proper key management should include:

Generation→Distribution→Storage→Use→Rotation/Expiration→Revocation
 $\text{Generation} \rightarrow \text{Distribution} \rightarrow \text{Storage} \rightarrow \text{Use} \rightarrow \text{Rotation/Expiration} \rightarrow \text{Revocation}$

Importance

If an attacker substitutes their public key for the intended recipient's key, confidential information could be encrypted for the attacker.

Similarly, compromise of a user's private key can undermine confidentiality and/or authenticity depending on how that key is used.

Result

RSA can support secure email through public-key operations, but effective security requires **secure key generation, authenticated public-key distribution, private-key protection, certificate validation, and proper key lifecycle management.**

Q5. Cryptography for IoT Devices

Scenario Analysis

A smart-home system contains low-power IoT devices such as sensors, smart locks, lights, cameras, and controllers.

These devices require secure communication but have limited hardware resources.

The available choices are:

RSA\boxed{\text{RSA}}RSA

and

ECC\boxed{\text{ECC}}ECC

A. Constraints of IoT Devices

IoT devices commonly have several limitations.

1. Limited Processing Power

Many devices use low-power microcontrollers that cannot efficiently perform expensive cryptographic operations.

2. Limited Memory

IoT devices often have small amounts of RAM and flash storage.

Large cryptographic keys, certificates, and intermediate calculations can increase memory requirements.

3. Limited Battery Power

Many smart-home sensors operate using batteries.

Cryptographic operations consume CPU time and therefore energy.

4. Limited Network Bandwidth

IoT networks may use low-bandwidth wireless protocols.

Large keys, certificates, and signatures can increase communication overhead.

5. Long Operational Lifetime

Some IoT devices remain installed for many years, making secure key management, software updates, and algorithm agility important.

B. Recommended Cryptographic Approach

For the given RSA-versus-ECC choice:

ECC is recommended

ECC provides strong public-key security with significantly smaller keys than RSA at comparable classical security levels.

For example, a commonly cited modern comparison is approximately:

256-bit ECC $\approx$ 3072-bit RSA

for roughly comparable classical security strength.

RSA vs ECC

Feature	RSA	ECC
Key size for comparable security	Larger	Smaller
Storage requirements	Higher	Lower
Communication overhead	Higher	Lower
Suitability for constrained devices	Moderate/Poor	High
IoT applicability	Limited by resources	Well suited

Why ECC?

Smaller keys generally reduce:

- Memory requirements
- Certificate/signature sizes
- Communication overhead

- Storage requirements

ECC-based approaches such as **ECDH** can be used for key agreement, while **ECDSA** or other appropriate elliptic-curve signature schemes can provide authentication.

After key establishment, efficient symmetric encryption should be used for the actual IoT data.

A typical design is:

ECC Key Exchange → Symmetric Session Key → Authenticated Encryption
 $\boxed{\text{ECC Key Exchange}} \rightarrow \text{Symmetric Session Key} \rightarrow \text{Authenticated Encryption}$

This avoids using computationally expensive public-key cryptography for every data packet.

Result

ECC is the better choice for the given IoT scenario
 $\boxed{\text{ECC is the better choice for the given IoT scenario}}$

because IoT devices benefit significantly from smaller public keys and reduced storage and communication overhead while maintaining strong security when modern curves and protocols are correctly implemented.

Overall Conclusion

The five scenarios demonstrate that cryptographic security depends not only on selecting an encryption algorithm but also on selecting the correct **mode of operation, key-exchange mechanism, authentication method, and key-management strategy**.

For repeated data, ECB should be avoided because it exposes patterns; CBC is better for the classroom comparison, while authenticated encryption such as AES-GCM is preferable in modern systems. Among DES, 3-DES, and Blowfish, Blowfish is the strongest practical choice for the stated comparison, although AES is preferable for new banking systems. Diffie-Hellman requires authentication to prevent MITM attacks. RSA can support secure email through hybrid encryption and digital signatures, provided keys are managed securely. Finally, ECC is particularly suitable for resource-constrained IoT devices because it provides strong public-key security with substantially smaller keys than RSA.